

Explicación de JwtAuthenticationFilter.java

1. Introducción

El JwtAuthenticationFilter es el componente encargado de autenticar al usuario en cada petición HTTP utilizando el token JWT. No realiza el login, sino que valida el token en cada request posterior y establece la identidad en Spring Security.

2. Tipo de clase

Extiende **OncePerRequestFilter**, por lo que se ejecuta una única vez por cada petición HTTP dentro de la cadena de filtros de Spring Security.

3. Dependencias

JwtService: extrae y valida el token.

UserDetailsService: carga el usuario desde la base de datos.

HandlerExceptionResolver: gestiona errores devolviendo respuestas HTTP correctas.

4. Flujo del método doFilterInternal

1) Leer cabecera Authorization.

```
final String authHeader = request.getHeader("Authorization");
```

2) Comprobar si comienza por Bearer.

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Si NO hay token → deja pasar

```
if (authHeader == null || !authHeader.startsWith("Bearer ")) {
```

```
    filterChain.doFilter(request, response);
```

```
    return;
```

```
}
```

El filtro NO bloquea nada, solo nos dice que no hay usuario autenticado todavía, posteriormente Spring bloqueará si el endpoint requiere autenticación.

3) Extraer token JWT.

```
final String jwt = authHeader.substring(7);
```

Esto quita el "Bearer " de la cabecera (7 caracteres)

4) Obtener username/email del token.

```
final String userEmail = jwtService.extractUsername(jwt);
```

El JWT contiene claims (datos declarados dentro del token, normalmente pares clave-valor):

```
{
  "sub": "usuario@correo.com",
  "exp": 1712345678
}
```

Aquí se obtiene el sub (subject → username/email).

5) Verificar si el usuario ya está autenticado.

```
Authentication authentication =  
SecurityContextHolder.getContext().getAuthentication();
```

Si ya existe autenticación:

```
if (userEmail != null && authentication == null)
```

6) Cargar usuario desde base de datos.

```
UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);
```

Spring tiene ahora tanto el usuario del token como el real (BD)

7) Validar token (firma, expiración, integridad).

```
if (jwtService.isTokenValid(jwt, userDetails))
```

Debe comprobar: firma criptográfica, expiración, integridad que pertenece a ese usuario

8) Crear UsernamePasswordAuthenticationToken. Este usuario ya ha demostrado quién es.

- No necesita contraseña."
- NO valida password
- NO hace login
- Solo crea la sesión de seguridad en memoria

```
UsernamePasswordAuthenticationToken authToken =  
    new UsernamePasswordAuthenticationToken(  
        userDetails,  
        null,  
        userDetails.getAuthorities()  
    );
```

9) Guardar autenticación en SecurityContext.

```
authToken.setDetails(new  
WebAuthenticationDetailsSource().buildDetails(request));  
  
SecurityContextHolder.getContext().setAuthentication(authToken);
```

10) Continuar la cadena de filtros.

5. Momento clave de autenticación

La autenticación real ocurre cuando se ejecuta

`SecurityContextHolder.getContext().setAuthentication(authToken)`. A partir de ese instante Spring considera autenticado al usuario.

6. Objetivo del filtro

Convertir el token JWT enviado por el cliente en un usuario autenticado dentro del contexto de seguridad sin usar sesiones.

Explicación de SecurityConfiguration.java

Es la clase que define toda la política de seguridad de la API.

1. Declaración de la clase

@Configuration

@EnableWebSecurity

public class SecurityConfiguration

- @Configuration → Clase que define Beans de Spring.
- @EnableWebSecurity → Activa el sistema de Spring Security.

Desde este momento, todas las peticiones pasan por el sistema de seguridad.

2. Dependencias

private final AuthenticationProvider authenticationProvider;

private final JwtAuthenticationFilter jwtAuthenticationFilter;

◆ **AuthenticationProvider:** Es el componente que valida usuario y contraseña durante el login. Normalmente es un DaoAuthenticationProvider que:

- Usa UserDetailsService
- Usa PasswordEncoder

◆ **JwtAuthenticationFilter:** Es el filtro que valida el token en cada request posterior.

3. Método principal: SecurityFilterChain

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception

Este método define TODAS las reglas de seguridad.

3.1 Desactivar CSRF

.csrf(csrf -> csrf.disable())

CSRF protege aplicaciones con sesiones y cookies.

Lo desactivamos por que nuestra API es STATELESS + JWT, esto quiere decir que no usa sesiones, por lo tanto no necesitamos CSRF.

3.2 Definir qué rutas son públicas

```
.requestMatchers("/auth/**").permitAll()
```

Significa que las URI siguientes no necesitan autenticación, es muy importante por que si no, nadie podría loguearse (al necesitar login):

✓ /auth/login

✓ /auth/signup

3.3 El resto requiere autenticación

```
.anyRequest().authenticated()
```

Todo endpoint que NO empiece por /auth/ necesita un token válido, en caso contrario, nos proporcionará un:

401 Unauthorized

3.4 Modo Stateless (CRÍTICO)

```
.sessionManagement(session ->  
    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)  
    )
```

Esta parte nos está indicando que:

- El servidor NO guarda sesiones
- No hay HttpSession
- No hay cookies de sesión
- Cada petición debe traer su JWT

Es obligatorio cuando trabajas con JWT.

3.5 Registrar AuthenticationProvider

```
.authenticationProvider(authenticationProvider)
```

Esto indica a Spring: “Cuando alguien haga login, usa este proveedor para validar usuario y password”.

¿Cómo sería entonces el flujo de ejecución?

POST /auth/login

↓

AuthenticationManager

↓

AuthenticationProvider

↓

UserDetailsService

↓

PasswordEncoder

3.6 Insertar el filtro JWT

.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

Spring tiene un filtro clásico de login: UsernamePasswordAuthenticationFilter

Pero nosotros estamos usando JWT. Esto implica que insertamos nuestro filtro ANTES. Porque si el token es válido:

- No necesitamos login
- Ya podríamos autenticar al usuario directamente

Por lo tanto nosotros tenemos este orden real:

JwtAuthenticationFilter

↓

UsernamePasswordAuthenticationFilter

↓

Resto de filtros

4. Configuración CORS

@Bean

CorsConfigurationSource corsConfigurationSource()

Configura qué frontend puede llamar a la API.

configuration.setAllowedOrigins(List.of("http://localhost:8005"));

Solo permitirá peticiones desde ese origen.

configuration.setAllowedMethods(List.of("GET", "POST"));

Solo métodos GET y POST.

configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));

permitirá que el frontend envíe:

- Authorization (JWT)
- Content-Type

5. Flujo completo real del sistema

LOGIN

POST /auth/login

↓

AuthenticationProvider

↓

Usuario válido

↓

JWT generado

↓

Se devuelve al cliente

6. Ejemplo de petición protegida

GET /clientes

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

↓

JwtAuthenticationFilter

↓

Token válido

↓

SecurityContext

↓

Controller

Sin token:

401 Unauthorized